TU Wien
Institut für Logic and Computation
Forschungsbereich Algorithms and Complexity

186.866 Algorithmen und Datenstrukturen VU

Programmieraufgabe P1

PDF erstellt am: 26. März 2026

1 Vorbereitung

Um diese Programmieraufgabe erfolgreich durchführen zu können, müssen folgende Schritte umgesetzt werden:

1. Laden Sie das zugehörige Java-Projekt `P1.zip` aus TUWEL runter.
2. Entpacken Sie `P1.zip` und öffnen Sie das entstehende Projektverzeichnis, d.h. das *äußere* Verzeichnis `task`, in Ihrer Entwicklungsumgebung.
3. Öffnen Sie die folgende Datei in Ihrem Editor: `task/src/main/java/StudentSolutionForExerciseImplementation.java`
   In dieser Datei sind sämtliche Programmiertätigkeiten durchzuführen. Ändern Sie keine anderen Dateien im Projekt und fügen Sie auch keine neuen hinzu.
4. Füllen Sie Vorname, Nachname und Matrikelnummer in der Methode `StudentInformation provideStudentInformation()` aus.
5. Gehen Sie sicher, dass Java-Version 21 auf Ihrem System verfügbar ist.

   Alternativ zu einer manuellen Installation, können Sie in der Datei `settings.gradle` das auskommentierte Code-Snippet für eine automatische Installation nutzen.
6. Details zur Ausführung des Codes finden Sie in Abschnitt 6.

2 Hinweise

Einige Hinweise, die Sie während der Umsetzung dieser Aufgabe beachten müssen:

- Lösen Sie die Aufgaben selbst. Verwenden Sie auch keine (KI-)Tools z. B. zur Codegenerierung.
- Sie dürfen beliebig viele Hilfsmethoden schreiben und benutzen. Beachten Sie aber, dass Sie nur die oben geöffnete Datei abgeben und diese Datei mit dem zur Verfügung gestellten Framework lauffähig sein muss.
- Es ist grundsätzlich erlaubt, Bibliotheken wie z. B. das Java Collections Framework und die Java Math Klasse zu verwenden. Die Voraussetzung dabei ist, dass die Verwendung die eigentliche Programmieraufgabe nicht abnimmt. Beachten Sie, dass nur die oben genannte Java-Version auf unserem Testsystem zur Verfügung steht.

3 Übersicht

Die folgenden Aufgaben dienen vordergründig der Eingewöhnung an das Java-Framework. Aufgabe ist es unter anderem, eine bekannte Methode zur Determinantenberechnung zu implementieren. Zunächst soll in Abschnitt 5.1 Vertrautheit mit der Matrix-Vektor-Multiplikation erlangt werden. Danach implementieren Sie in Abschnitt 5.2 Basiskomponenten (z. B. Inversionszähler) und kombinieren diese in Abschnitt 5.3.

4 Theorie

Die Theorie für diese Programmieraufgabe befindet sich in den Vorlesungsfolien „Analyse von Algorithmen“.

5 Implementierung

5.1 Matrix-Vektor-Multiplikation

Gegeben ist eine quadratische Matrix $A \in \mathbb{R}^{n \times n}$ und ein Spaltenvektor $b \in \mathbb{R}^n$. Betrachten Sie das folgende Gleichungssystem.

$$\begin{array}{rcl} a_{00}x_0 + a_{01}x_1 + \quad \cdots \quad + a_{0,n-1}x_{n-1} & = & b_0 \\ a_{10}x_0 + a_{11}x_1 + \quad \cdots \quad + a_{1,n-1}x_{n-1} & = & b_1 \\ & \vdots & \\ a_{n-1,0}x_0 + a_{n-1,1}x_2 + \quad \cdots \quad + a_{n-1,n-1}x_{n-1} & = & b_{n-1} \end{array} \tag{1}$$

Sie sollen für gegebene $A = (a_{ij})_{i,j=0}^{n-1} \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$ und eine Kandidatenlösung $x = (x_i)_{i=0}^{n-1} \in \mathbb{R}^n$ prüfen, ob x eine gültige Lösung des Gleichungssystems (1) ist. Matrizen speichern wir als Array von Arrays, sodass für eine als `int[][] A` gespeicherte Matrix A der Zugriff auf a_{ij} via `A[i][j]` möglich ist.

Schreiben Sie dazu eine Methode `bool isFeasible(int[][] A, int[] b, int[] x)`, welche zurückgibt, ob die Matrix-Vektor-Multiplikation Ax den Vektor b ergibt.

5.2 Hilfsfunktionen zur Determinantenberechnung

Die *symmetrische Gruppe* S_n umfasst alle Permutationen einer n-elementigen Menge. Wir notieren diese als die Menge aller Tupel $s = (s_0, s_1, \ldots, s_{n-1})$ mit $s_i \in \{0, 1, \ldots, n-1\}$ für $i = 0, 1, \ldots, n-1$, sodass außerdem $s_i \neq s_j$ für alle $i \neq j$. Für $n = 5$ ist $(s_0, s_1, s_2, s_3, s_4) = (3, 4, 0, 2, 1)$ ein Beispielelement aus S_5.

Ein *Fehlstand* (oft auch *Inversion* genannt) einer Permutation s ist ein Paar $(i, j) \in \{0, 1, \ldots, n-1\}^2$, mit $0 \leq i < j \leq n-1$, sodass $s_i > s_j$. Schreiben Sie eine Methode `List<int[]> inversionStatistic(int[] permutation)`, welche die vollständige Liste der durch `int[2]`-Arrays repräsentierten Fehlstände zurückgibt – also ein Indexpaar (i, j) wie oben beschrieben. Die Reihenfolge der Liste kann hier beliebig sein. Beispielsweise würde für das Permutation-kodierende Array `{3,1,2,0}` die Liste bestehend aus den Länge-2 Arrays `{0,1}`, `{0,2}`, `{0,3}`, `{1,3}` und `{2,3}` zurückgegeben werden.

5.3 Leibniz-Formel

Eine Möglichkeit, $\det(A)$ zu berechnen ist über die *Leibniz-Formel*[1], welche besagt, dass

$$\det(A) = \sum_{s \in S_n} \operatorname{sgn}(s) \prod_{i=0}^{n-1} a_{i,s_i}, \tag{2}$$

wobei sich das *Signum* $\operatorname{sgn}(s)$ von s berechnen lässt durch die Formel

$$\operatorname{sgn}(s) := \begin{cases} +1 & \text{wenn die Anzahl der Fehlstände gerade ist,} \\ -1 & \text{wenn die Anzahl der Fehlstände ungerade ist.} \end{cases}$$

Verwenden Sie die in Abschnitt 5.2 vorbereiteten Konzepte, um die Methode `long Leibniz(int[][] A)` zu implementieren, welche die Determinante einer Matrix, repräsentiert durch das Array `A`, gemäß der Formel (2) berechnet.

Zu gegebener $n \times n$-Matrix können Sie davon ausgehen, dass die Berechnung von S_n (Permutationen Ihrer Zeilen-/Spaltenindices) für Sie vorbereitet ist. Sie kann nach dem Import
```
import exercise.SymmetricGroup;
```
mit
```
int[][] SG = (new SymmetricGroup(n)).getSymmetricGroup();
```
aufgerufen werden.

Hinweis. Führen Sie die Summation iterativ durch; die speichernde Variable sollte vom Typ `long` sein, da wir mitunter sehr große Zahlenbereiche antreffen.

6 Testen

Führen Sie im Projektverzeichnis den Befehl `./gradlew run` aus (`.\gradlew.bat run` unter Windows).

Sollten Sie IntelliJ nutzen, können Sie das Projekt auch direkt über die grafische Oberfläche starten. Klicken Sie dazu in der rechten Leiste auf das Gradle-Logo (Elefant) und navigieren Sie zum „run“-Task (siehe Abbildung 1). Mittels Rechtsklick bzw. Sekundärklick können Sie das Projekt ausführen („Run“) oder auch den visuellen Debugger nutzen („Debug“). Die Option „Modify Run Configuration…“ erlaubt es Ihnen, die Ausführung permanent als „Run/Debug Configuration“ zu hinterlegen.

[1] Diese dient oft auch als Definition der Determinante.

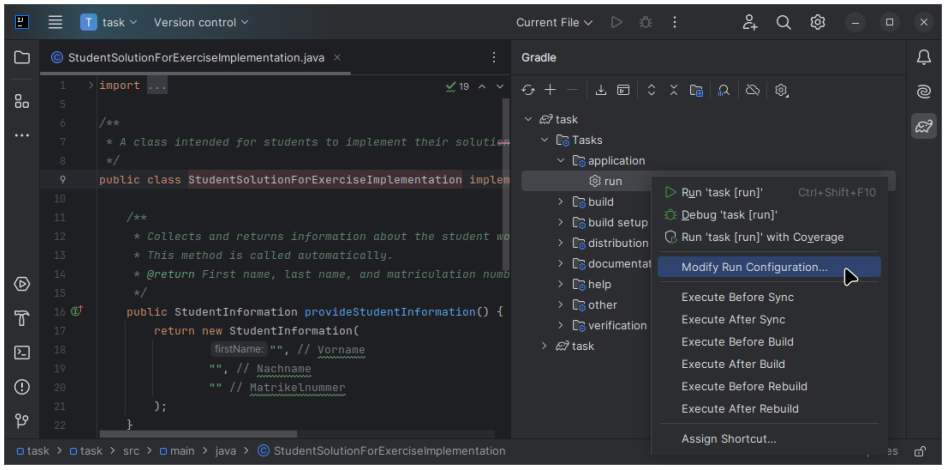

Abbildung 1: Ausführen von Gradle-Tasks in IntelliJ

Anschließend wird Ihnen in der Konsole eine Auswahl an Testinstanzen angeboten, darunter befindet sich zumindest `abgabe.csv`:

```
Select an instance set or exit:
[1] abgabe.csv
[0] Exit
```

Durch die Eingabe der entsprechenden Ziffer kann entweder eine Testinstanz ausgewählt werden oder das Programm (mittels der Eingabe von `0`) verlassen werden. Wird eine Testinstanz gewählt, dann wird der von Ihnen implementierte Programmcode ausgeführt. Kommt es dabei zu einem Fehler, wird ein Hinweis in der Konsole ausgegeben.

Um nur eine bestimmte Subinstanz auszuführen, hängen Sie `:[Subinstanznummer]` an Ihre Auswahl an (z. B. `1:123`).

Relevant für die Abgabe ist das Ausführen der Testinstanz `abgabe.csv`.

Die Testinstanz `abgabe.csv` sollte in höchstens ein paar Minuten durchlaufen, andernfalls ist etwas ineffizient implementiert.

7 Evaluierung

Wenn der von Ihnen implementierte Programmcode mit der Testinstanz `abgabe.csv` ohne Fehler ausgeführt werden kann, dann wird nach dem Beenden des Programms im Verzeichnis `results` eine Ergebnisdatei mit

dem Namen `solution-abgabe.csv` erzeugt. Beachten Sie dabei, dass Sie die gesamte Testinstanz ausführen und nicht nur eine Subinstanz.

Die Datei `solution-abgabe.csv` beinhaltet Zeitmessungen der Ausführung der Testinstanz `abgabe.csv`, welche in einem Webbrowser visualisiert werden können. Auch Ergebnisdateien anderer Testinstanzen können zu Testzwecken visualisiert werden. Öffnen Sie dazu die Datei `visualization.html` in Ihrem Webbrowser und klicken Sie rechts oben auf den Dateiauswahlknopf, um `solution-abgabe.csv` auszuwählen.

Beantworten Sie basierend auf der Visualisierung die Fragestellungen aus dem folgenden Abschnitt.

Beachten Sie, dass in den Plots für eine etwas stärkere Aussagekraft zu jeder Dimension die Laufzeiten von *zehn Instanzen* mit dieser Dimension dargestellt werden.

8 Fragestellungen

Öffnen Sie `solution-abgabe.csv` und bearbeiten Sie folgende Aufgaben- und Fragestellungen:

1. Durch Klicken auf Gruppennamen in der Legende neben den Plots lassen sich einzelne Gruppen aus- bzw. einblenden. Betrachten Sie zuerst den oberen Plot, der die Laufzeit Ihres Algorithmus zur Matrix-Vektor-Multiplikation bei wachsendem n zeigt. Beschreiben Sie, wie die Laufzeit von n abhängt. Wie erklären Sie sich diese Laufzeitabhängigkeit? Kommentieren Sie die Unterschiede, die durch Wahl der Gruppe *Gültigkeit Lösung – false* bzw. *Gültigkeit Lösung – true* im Scatterplot ersichtlich werden. Speichern Sie im Anschluss die Plots zu den drei möglichen Gruppen-Auswahlen als Bild, indem Sie in der Menüleiste rechts über dem Plot auf den Fotoapparat drücken.

2. Betrachten Sie nun den mittleren Plot, der die Laufzeit Ihres Algorithmus zur Enumeration aller Fehlstände einer Permutation bei wachsender Permutationslänge n zeigt. Beschreiben Sie, wie die Laufzeit von n abhängt. Wie erklären Sie sich diese Laufzeitabhängigkeit? Speichern Sie im Anschluss den Plot als Bild.

3. Betrachten Sie nun den unteren Plot, der die Laufzeit der Determinantenberechnung mittels der Leibniz-Formel für

quadratische $n \times n$ Matrizen zu wachsendem n zeigt. Welche asymptotische Laufzeitabhängigkeit von der Anzahl der Mengen lässt sich erkennen? Entspricht das Ihren Erwartungen? Warum bzw. warum nicht? Argumentieren Sie möglichst genau. Speichern Sie im Anschluss den Plot als Bild.

Falls sich im Zuge der Evaluierung die Darstellung der Plots auf ungewünschte Weise verändert (z. B. durch die Auswahl eines zu kleinen Ausschnitts), können Sie mittels Doppelklick auf den Plot oder Klick auf das Haus in der Menüleiste die Darstellung zurücksetzen.

Fügen Sie Ihre Antworten in einem Bericht gemeinsam mit den erstellten Bildern der Visualisierung der Laufzeiten der Testinstanz `abgabe.csv` zusammen.

9 Abgabe

Laden Sie die Datei `task/src/main/java/StudentSolutionForExerciseImplementation.java` in der TUWEL-Aktivität *Hochladen Source-Code P1* hoch. Fassen Sie diesen Bericht mit den anderen für das zugehörige Abgabegespräch relevanten Berichten in einem PDF zusammen und geben Sie dieses in der TUWEL-Aktivität *Hochladen Bericht Abgabegespräch 1* ab.